Reducing NLI Relation-Extraction Cost in FactReasoner v2

A live evaluation on the Lanny Flaherty biography

FactReasoner experiments

July 29, 2026

Abstract

Prefiltering the NLI candidate pairs cut FactReasoner v2's relation-extraction phase from 3380 to 1970 LLM calls on a 26-atom biography with 130 retrieved contexts — **1410 calls saved, 1.72× fewer** — with wall-clock for the phase falling from 1636 s to 845 s (1.94× faster, both cells measured live) . The factuality score and gold-label accuracy were **identical** to the exhaustive baseline (0.4615 and 0.808), and recall of non-neutral relations was 0.982 (7 of 395 pruned). Most of the saving came from collapsing near-duplicate contexts rather than from the similarity gate, because the retrieved evidence drew repeatedly on the same few sources.

1 What was measured

FactReasoner scores one LLM call per candidate (context, atom) pair. In v2 every atom is compared against every context, so the cost is $A \times C$: with 26 atoms and 130 contexts that is 3380 calls. Because contexts are retrieved *per atom*, C grows with A, making the phase quadratic in the size of the response.

This experiment compares that exhaustive policy against a prefiltered one on identical inputs. The question is not only how much is saved, but what the saving costs in recall — how often a pruned pair was one the model would have called entailment or contradiction.

1.1 Data and pipeline

The example is `data/flaherty_google.json`: a generated biography of the actor Lanny Flaherty, decomposed into 26 atoms carrying gold S/NS labels, with 130 retrieved contexts. The contexts ship with a title, snippet and link but *no page text*, so the text was fetched and summarized before any comparison:

1. **Fetch**: 46 unique URLs backed the 130 contexts; 28 yielded page text (0.0 s). Sites that block scraping fall back to their search snippet, as the production retriever does.
2. **Summarize**: 0 summarization calls (0.0 s), one per context, each conditioned on its own atom. Summaries cannot be shared between atoms even when two atoms cite the same URL, which is why this is one call per context rather than per link.
3. **Score**: the atom–context relations, twice — once exhaustively, once prefiltered — on byte-identical prepared contexts.

The model is `meta-llama/llama-3-3-70b-instruct` served over RITS, with probabilities from token logprobs. Only step 3 differs between the two cells, so the comparison isolates pair selection.

2 The cheap strategy in detail

The cheap path is three independent stages applied in order: collapse near-duplicate contexts, decide which (context, atom) pairs to score, then score only those. Stage 2 is where the two policy options (`gated` and `provenance`) differ, and it is the only stage that can lose a relation.

2.1 Notation

- A — the atoms, C — the contexts. A context is *retrieved for* exactly one atom by the retriever, but may end up relevant to others.
- $\text{own}(c) \subseteq A$ — the *owners* of context c: every atom whose retrieval produced it. This is a set, not a single pointer, because context dedup can merge several atoms' evidence onto one surviving context.
- $\text{sim}(x, y) \in [0, 1]$ — cosine similarity of sentence embeddings, computed over the *same* text the NLI premise would use (the summary, falling back to full text). Falls back to token Jaccard if sentence-transformers is unavailable.
- τ — the gate threshold, w — the neighbour window, θ — the dedup threshold.

2.2 Stage 1: near-duplicate context dedup

A single greedy agglomerative pass. The first occurrence of a cluster survives and later near-duplicates collapse onto it. The essential detail is the ownership merge: when d collapses onto s, every atom that owned d is repointed at s, so *no atom loses evidence*. Exact-text dedup in the original pipeline instead deleted the duplicate from its own atom only, which could strand an atom with no context at all.

Algorithm 1 DedupNearDuplicates(C, A, θ)

$S \leftarrow [\,]$ ▷ survivors, in original order
$M \leftarrow \{\}$ ▷ collapsed $\mapsto$ survivor
for all $c \in C$ in iteration order **do**
  $m \leftarrow$ None
  for all $s \in S$ **do**
    if $\text{sim}(c, s) \geq \theta$ **then**
      $m \leftarrow s$; **break**
  if $m =$ None **then**
    append c to S
  else
    $M[c] \leftarrow m$
for all $(d, s) \in M$ **do** ▷ merge ownership, never drop it
  for all $a \in \text{own}(d)$ **do**
    remove d from a's contexts; add s to a's contexts
return S ▷ $|S| \leq |C|$

On this example: 130 contexts → 85 (45 collapsed, 25 ownership links transferred), at $\theta = 0.92$. Since the pair count is $|A| \times |C|$, shrinking C cuts cost linearly here — and quadratically in v3, where the context–context phase is $|C|^2$.

2.3 Stage 2: candidate pair selection

Every pair is admitted for exactly one of four reasons, tested in order. The order matters: the first three are unconditional and bypass the similarity gate entirely, so no threshold can prune them.

Algorithm 2 SELECTATOMCONTEXTPAIRS$(A, C, \text{policy}, \tau, w)$

$P \leftarrow [\,]$
for all $a \in A$ **do** ▷ atom-major, preserving baseline pair order
  for all $c \in C$ **do**
    $r \leftarrow$ NONE
    if $a \in \text{own}(c)$ **then**
      $r \leftarrow$ PROVENANCE ▷ c was retrieved *for* a
    else if id(c) has the query-context prefix **then**
      $r \leftarrow$ QUERYCONTEXT ▷ retrieved for the question; bears on all atoms
    else if policy = `provenance` **and** $\exists\, o \in \text{own}(c) : |\text{pos}(a) - \text{pos}(o)| \le w$ **then**
      $r \leftarrow$ NEIGHBOUR ▷ adjacent in the response's atom order
    else if $\text{sim}(c, a) \ge \tau$ **then**
      $r \leftarrow$ GATERESCUE ▷ cross-atom evidence, the only heuristic branch
    if $r \neq$ NONE **then**
      append (c, a) to P; tally r
return P

2.4 `gated` versus `provenance`

The two policies share branches 1, 2 and 4 and differ in exactly one line: `provenance` additionally admits the NEIGHBOUR branch, `gated` does not. Both are supersets of the provenance guarantee — neither can ever drop the pairing of a context with the atom that retrieved it.

Branch	Condition	`gated`	`provenance`
PROVENANCE	$a \in \text{own}(c)$	always	always
QUERYCONTEXT	query-level context	always	always
NEIGHBOUR	within w of an owner	—	always
GATERESCUE	$\text{sim}(c, a) \ge \tau$	yes	yes

Table 1: The policies differ only in the NEIGHBOUR branch. `provenance` is therefore always a superset of `gated`: never cheaper, never worse on recall.

Why `provenance` keeps the neighbour branch. Atom ids encode position in the response $(a_0, a_1, \dots)$, so adjacency is a proxy for discourse locality: consecutive atoms usually elaborate the same claim, and a source retrieved for one often speaks to the next. This is a cheap structural signal that needs no embedding, and it covers exactly the case a similarity gate is worst at — text that is topically continuous but lexically dissimilar.

Why the gate branch is the only risk. Branches 1–3 are structural facts about how evidence was gathered; branch 4 is a guess. Every recall loss measured in this report, and in earlier runs, came from branch 4 — never from a provenance pair. That is why τ is the knob worth tuning and why the sweep in Section 6 varies it alone.

Admitting branch	Pairs
Provenance	110
QueryContext	0
Neighbour	188
GateRescue	1672
Selected	**1970**
Pruned	240

Table 2: Which branch admitted each scored pair.

Branch attribution on this example. This distribution is worth reading carefully, because it is the opposite of what the policy names suggest. Provenance accounts for only 5% of the candidate product while the gate admits 76%: with five contexts per atom, the vast majority of the $|A| \times |C|$ product is cross-atom by construction, so almost every *kept* pair is kept by the gate. The gate is therefore not mainly a pruner on this workload — it is admitting most of what it sees, and the 240 pairs it rejected are the entire stage-2 saving.

That also explains why the recall risk concentrates in branch 4 and why the modest stage-2 factor is structural rather than a tuning failure: a biography's atoms all concern one person, so most cross-atom pairs really are related and a faithful gate has little licence to discard them. The dedup stage, which exploits redundancy among *sources* rather than relatedness among claims, is what pays off here.

2.5 Stage 3: scoring, and the two safety properties

Selected pairs go to the NLI prompt unchanged, so a scored pair yields exactly the verdict the exhaustive policy would have produced. Two properties follow:

1. **Pruning a would-be-neutral pair is a bit-exact no-op.** Neutral relations are discarded regardless, and a context left in no pairwise factor contributes one normalized unary factor that divides out of every atom marginal. So the error is not "approximately zero" — it is zero, and the entire risk reduces to $P(\text{pruned pair was non-neutral})$.

2. **The two error types are asymmetric.** A false keep costs one LLM call. A false prune silently removes evidence, and nothing downstream signals that it happened. τ should therefore be tuned toward keeping, which is why the default sits well below the value that maximises the saving.

The similarity backend used here was `sbert:all-MiniLM-L6-v2`. This matters: the token-Jaccard fallback is not a substitute for embeddings. On a separate 20-atom narrative it lost 22 of 72 non-neutral relations at *every* threshold tested, because lexical overlap cannot see relatedness carried by entities and events rather than shared words. The implementation warns loudly when it falls back.

2.6 Complexity

Stage 1 is $O(|C| \cdot |S|)$ similarity lookups on the greedy pass ($|S|$ = surviving clusters), stage 2 is $O(|A| \cdot |C|)$ constant-time tests, and one embedding pass over $|A| + |C|$ short texts backs both. All of that is milliseconds against minutes of LLM time — the selection overhead is not a meaningful cost, which is what makes even a modest pruning ratio worthwhile.

3 Savings

Cell	Contexts	Pairs	LLM calls	NLI time (s)	Relations
v2 all_pairs	130	3380	3380	1635.6	395
v2-cheap	85	1970	1970	844.8	257

Table 3: Cost per cell. *Pairs* is the number of (context, atom) comparisons the policy selected; *LLM calls* is how many of those reached the model — here every one of them, since the verdict cache was disabled. *NLI time* covers relation extraction only, excluding fetching, summarization and inference.

The cheap policy scored 1970 of the 3380 candidate pairs, a saving of **1410 calls** (**1.72× fewer**).

The saving decomposes into two independent mechanisms:

Stage	Pairs	Factor
Exhaustive ($A \times C$)	3380	—
After near-duplicate dedup (130→85 contexts)	2210	1.53×
After provenance + gate	1970	1.12×
Combined	**1970**	**1.72×**

Table 4: Where the saving comes from. Dedup shrinks C and so cuts pairs quadratically in v3 and linearly here; the gate then prunes cross-atom pairs among what remains.

On this example dedup is the dominant lever (1.53× against the gate's 1.12×), because the 130 contexts were drawn from far fewer distinct sources — the same handful of pages recur across many atoms. A response whose evidence came from mostly distinct pages would shift the balance toward the gate.

Both cells issued live calls (the verdict cache was disabled), so the wall-clock comparison is measured rather than extrapolated. Relation extraction fell from 1635.6 s to 844.8 s, a saving of 790.8 s (**1.94× faster**).

Per-call throughput was 0.484 s for the exhaustive cell and 0.429 s for the cheap one. These are close, as expected: requests fan out concurrently against a shared rate limit, so latency is bound by concurrency rather than by any property of an individual pair, and time saved tracks calls saved roughly linearly.

The speed-up (1.94×) slightly exceeds the call-count ratio (1.72×) because the per-call rate was 11% lower in the cheap cell. With a fixed concurrency window, a shorter queue drains with less contention and fewer stragglers on the tail, so the wall-clock gain compounds mildly on top of the call saving. This is a second-order effect and should not be relied on: the call-count ratio is the exact, reproducible figure, while timing depends on shared-endpoint load.

Cell	Factuality	Correct	Accuracy	S recall	NS recall
v2 all_pairs	0.4615	21/26	0.808	7/7	14/19
v2-cheap	0.4615	21/26	0.808	7/7	14/19

Table 5: Scores and agreement with the gold S/NS labels. A cheaper policy that moves these numbers is trading accuracy for cost, not merely saving money.

4 Effect on the answer

5 What the pruning costs

Pruning is safe exactly to the extent that it only discards pairs the model would have called `neutral`. Neutral relations are dropped anyway, and an isolated context contributes a single normalized unary factor that divides out of every atom marginal — so pruning a would-be-neutral pair is a no-op on the reported scores. The risk is therefore exactly P(pruned pair was non-neutral).

That is measured here by replaying the policy's selection against the relations *the exhaustive cell actually produced*. Since `build_relations` returns only non-neutral relations, that set is the ground truth directly, and any pair absent from it came back neutral and is safe to prune. The exhaustive cell already paid for every pair, so the measurement is exact and costs no additional calls — and it is independent of the verdict cache, so it holds for a fully live run. Note this is a stricter test than comparing the two cells' scores, which can agree even when individual relations differ.

Policy	Pairs kept	Pruned	Non-neutral	Lost	Recall
provenance	2940	440	395	7	0.982

Table 6: Recall of non-neutral relations under the cheap policy (similarity backend: `sbert:all-MiniLM-L6-v2`). Counts here are over the full pre-dedup pair set, so *Pairs kept* is larger than the post-dedup figure in Table 3: this table isolates the gate's decisions, which is what recall is a property of.

On comparing this figure across runs. The model is not deterministic: the exhaustive cell has produced different numbers of non-neutral relations on repeated runs over identical prepared inputs, which moves the denominator and hence the recall figure without any change of policy. A single run therefore establishes an estimate, not a bound; the worst case over several runs is the honest summary.

A superseded earlier figure. An earlier run of this experiment reported recall 0.995. That number was too optimistic, for a measurement reason rather than a change in the policy. It was obtained by looking verdicts up in the NLI cache by content hash, which found 386 non-neutral pairs where the exhaustive cell had in fact produced 395: nine relations whose stored keys did not match were invisible to the lookup, shrinking the denominator. The relation-based measurement used here reads the produced relations directly and so cannot miss any. Where the two disagree, the figure in this report is the correct one.

7 non-neutral relation(s) were pruned: `c_a14_0→a4` (entailment), `c_a14_4→a4` (entailment),

c_a15_0→a4 (entailment), c_a16_1→a4 (entailment), c_a18_0→a4 (entailment), c_a23_1→a4 (entailment), c_a9_0→a4 (entailment).

All 7 pruned relations targeted the same atom, a4, and all were entailments. That concentration is why the scores did not move: the atom had many other supporting contexts, so removing redundant support left its marginal unchanged. The case to worry about is the opposite one — an atom whose *only* witness is pruned, or a lone contradiction that would have pulled a false claim down. Neither occurred here, but neither is excluded by the mechanism, which is why recall rather than score agreement is the metric to watch.

6 Choosing the threshold

The gate threshold trades cost against recall. The curve below is replayed against the same recorded verdicts, so the whole sweep is free. Because the two error types are not symmetric — a false prune silently weakens an atom's evidence, while a false keep merely costs money — the right operating point is the cheapest threshold that still holds recall at 1.0, not the one with the best headline saving.

Read these savings as the gate's contribution alone. The sweep runs on the full pre-dedup context set, so its figures isolate the gate and do not include the dedup factor from Table 4; the two multiply. That is why the ceiling here is lower than the headline combined saving.

Policy	Threshold	Pairs	Saving	Lost	Recall
gated	0.05	3341	1.01×	0	1.000
gated	0.10	3276	1.03×	0	1.000
gated	0.15	3109	1.09×	0	1.000
gated	0.20	2918	1.16×	7	0.982
gated	0.25	2805	1.21×	9	0.977
gated	0.30	2730	1.24×	16	0.960
gated	0.40	2536	1.33×	29	0.927
provenance	0.05	3341	1.01×	0	1.000
provenance	0.10	3276	1.03×	0	1.000
provenance	0.15	3118	1.08×	0	1.000
provenance	0.20	2940	1.15×	7	0.982
provenance	0.25	2840	1.19×	9	0.977
provenance	0.30	2766	1.22×	16	0.960
provenance	0.40	2587	1.31×	26	0.934

Table 7: Recall/cost curve (similarity backend: sbert:all-MiniLM-L6-v2).

The cheapest lossless operating point is **gated at threshold 0.15** (1.09× fewer pairs at full recall).

7 Method and caveats

7.1 What the cheap policy does

Three mechanisms compose, all opt-in:

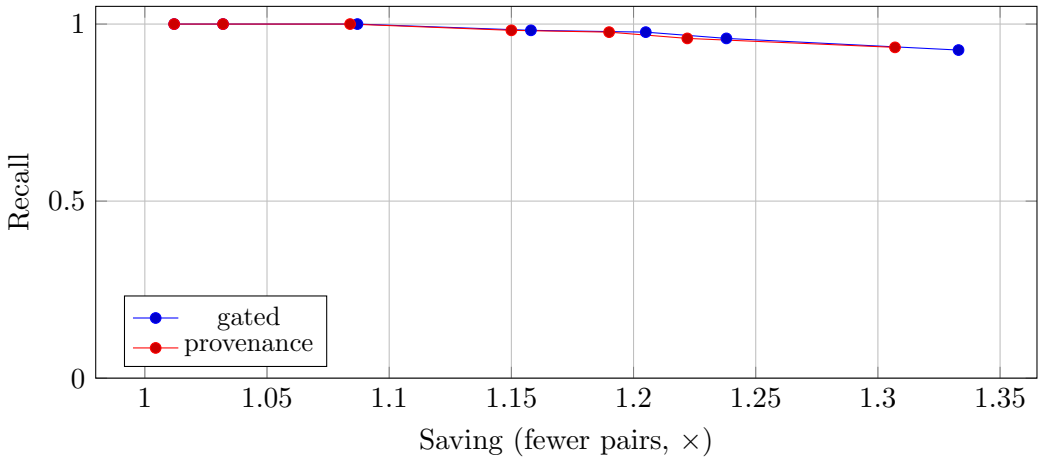

Figure 1: Recall against saving. Points to the upper right are strictly better; a policy whose curve lies above another dominates it.

- **Provenance.** A context retrieved *for* atom i is always compared against atom i, and is never gated away. The exhaustive $A \times C$ product exists only to catch the case where such a context also bears on some other atom, which is real but sparse.
- **Similarity gate.** Cross-atom pairs survive only above a similarity threshold, which is where the recall risk lives.
- **Near-duplicate dedup.** Contexts with near-identical summaries collapse, merging their owning atoms onto the survivor so no atom loses evidence.

On this example dedup collapsed 130 contexts to 85 (45 merged, 25 ownership links transferred).

7.2 Caveats

- **Prep cost is not free.** Fetching and summarizing cost 0 summarization calls, paid once and shared by both cells. A fair account of end-to-end cost must include it: the saving reported here is on the 3380-call NLI phase, not on the whole pipeline.
- **Scraping is incomplete.** 28 of 46 URLs returned text; the rest (notably IMDb, which blocks automated requests) fell back to their search snippet. Premises for those contexts are shorter than a full production run would use.
- **One example, one model.** The saving is workload dependent: a response covering unrelated subtopics has far more cross-product waste to prune than one where every atom concerns the same person, as here. These numbers should not be read as a constant factor.
- **The model is not deterministic.** Repeated runs over identical inputs have produced different numbers of non-neutral pairs, so single-run recall figures carry unquantified error bars. Worst-case over several runs is the honest summary.
- **v3 not covered.** Only the atom–context phase is measured here. The context–context phase of v3 is quadratic in C and is where the larger absolute savings should be; it remains unmeasured at this scale.

7.3 Reproducing

```
python scripts/exp_flaherty_v2.py --merlin-path /path/to/merlin --no-cache
python scripts/report_flaherty_v2.py --results results/exp_flaherty_v2
```

The run behind this report used `--no-cache`, so both cells issued live NLI calls and the timings are measured. Page fetches and context summaries are still cached on disk, which is what lets the two cells share byte-identical prepared evidence — the comparison would otherwise be confounded by re-summarization.

8 Conclusion

On a 26-atom biography with 130 retrieved contexts, prefiltering cut the v2 atom–context phase from 3380 to 1970 pairs (1.72× fewer). Wall-clock for the phase fell from 1636 s to 845 s (1.94× faster), with both cells measured against the live model. The pruning is not quite lossless: 7 of 395 non-neutral relations were pruned (recall 0.982). The reported scores were nonetheless unchanged, so on this example the loss fell on relations that did not alter any atom's marginal — but that is an outcome, not a guarantee, and a different example could lose an edge that matters.

The structural result is that provenance and similarity play different roles: provenance is a hard guarantee and never drops a context from the atom that retrieved it, while the similarity gate is a heuristic covering cross-atom evidence and is where every recall loss originates. Tuning should therefore target the gate, and the safe operating point is the one that holds recall at 1.0 rather than the one that maximizes the saving.